

ATIVIDADE DIAGNÓSTICA — 4ª FASE

Faxinator

Sistema de Agendamento de Serviços de Limpeza

Documentação do Projeto

Lista de Requisitos Funcionais

Descritivo de Casos de Teste de Software

Lista de Requisitos de Infraestrutura

Autor: Gustavo Fiabani

Disciplina de Implantação

Agosto de 2026

Sumário

1. Contextualização e Escopo
2. Requisitos de Infraestrutura
 - 2.1. Ambiente de desenvolvimento
 - 2.2. Dependências do projeto
3. Arquitetura da Solução
 - 3.1. Visão geral
 - 3.2. Organização do backend
 - 3.3. Organização do frontend
 - 3.4. Decisões técnicas relevantes
4. Modelo de Dados
5. Lista de Requisitos Funcionais
6. Descritivo de Casos de Teste de Software
 - 6.1. Ferramentas utilizadas
 - 6.2. Ambiente de teste
 - 6.3. Massa de dados
 - 6.4. Casos de teste
 - 6.5. Resumo dos resultados
 - 6.6. Defeitos encontrados e corrigidos
7. Instruções de Execução
8. Credenciais de Acesso

1. Contextualização e Escopo

Uma empresa de serviços de limpeza oferece agendamentos de faxinas residenciais e comerciais, mas enfrenta dificuldades na gestão desses agendamentos por não possuir um sistema informatizado para controlar datas e horários das solicitações. A ausência de controle resulta em agendamentos duplicados, falha de comunicação entre funcionários e clientes, e dificuldade em organizar a disponibilidade dos profissionais.

O **Faxinator** é um sistema web desenvolvido para atender essa necessidade. Ele permite ao operador da empresa agendar serviços de faxina, gerenciar horários e a disponibilidade dos profissionais de limpeza, e registrar todas as movimentações realizadas.

O sistema incorpora um mecanismo de controle de agendamentos que emite alertas automáticos em duas situações distintas: quando há *conflito de horário* com outro serviço já atribuído ao mesmo profissional, e quando o profissional *não possui disponibilidade cadastrada* para o dia e horário solicitados. Além disso, mantém um histórico completo de cada operação, identificando o cliente, o profissional responsável, o operador que executou a ação e a data da operação, garantindo rastreabilidade e transparência dos processos.

Entregas cobertas por este documento

Entrega	Descrição	Localização
1	Lista de requisitos funcionais	Seção 5 deste documento
2	Diagrama Entidade-Relacionamento (DER)	Arquivo de imagem em <code>der/</code> (entrega separada)
3	Script de criação e população do banco	<code>backend/db/faxina_db.sql</code>
4	Código-fonte do sistema	<code>backend/</code> e <code>frontend/</code>
8	Descritivo de casos de teste de software	Seção 6 deste documento
9	Lista de requisitos de infraestrutura	Seção 2 deste documento

2. Requisitos de Infraestrutura

2.1. Ambiente de desenvolvimento

Item	Tecnologia	Versão
SGBD	PostgreSQL	18.3 (PostgreSQL 18.3 on x86_64-windows, compiled by msvc-19.44.35225, 64-bit)
Linguagem de programação	TypeScript (backend)	5.9.3
	TypeScript (frontend)	6.0.3
	Node.js (ambiente de execução)	v24.11.1
	npm (gerenciador de pacotes)	11.12.1
Sistema operacional	Microsoft Windows 11 Pro	10.0.26200.8973

2.2. Dependências do projeto

Backend — API REST responsável pela regra de negócio e acesso ao banco de dados.

Pacote	Versão	Finalidade
express	5.2.1	Framework HTTP: roteamento, middlewares e tratamento de requisições
pg	8.23.0	Driver PostgreSQL, com consultas parametrizadas e pool de conexões
bcrypt	6.0.0	Geração e verificação de hash de senha (custo 12)
jsonwebtoken	9.0.3	Assinatura e verificação do token de acesso (JWT)
cookie-parser	1.4.7	Leitura do cookie httpOnly que transporta o token
cors	2.8.5	Liberação de origem cruzada com envio de credenciais
dotenv	17.3.1	Carregamento das variáveis de ambiente do arquivo <code>.env</code>
tsx	4.21.0	Execução direta de TypeScript em desenvolvimento

Frontend — Interface web (SPA) consumida pelo operador.

Pacote	Versão	Finalidade
react / react-dom	19.2.8	Biblioteca de construção da interface
vite	8.2.1	Servidor de desenvolvimento e empacotador
react-router	8.3.0	Roteamento entre as telas da aplicação
axios	1.19.0	Cliente HTTP para consumo da API, com envio de cookies

Pacote	Versão	Finalidade
tailwindcss	4.3.3	Estilização utilitária das interfaces
react-toastify	11.1.0	Notificações de sucesso das operações
react-icons	5.7.0	Conjunto de ícones

Requisitos mínimos para execução: PostgreSQL 18 ou superior instalado e em execução; Node.js 20 ou superior; navegador web moderno (Chrome, Edge ou Firefox em versão atual); portas 5000 (API) e 3000 (interface) livres.

3. Arquitetura da Solução

3.1. Visão geral

O sistema é dividido em dois projetos independentes que se comunicam por uma API REST:

- **backend** — API REST em Node.js com Express, acessando o PostgreSQL diretamente pelo driver `pg`, sem ORM. Concentra toda a regra de negócio, as validações e as verificações de conflito e disponibilidade.
- **frontend** — aplicação React de página única, responsável apenas pela apresentação e pela captura de dados do operador.

A autenticação usa **JWT transportado em cookie httpOnly** com expiração de 1 hora. Como o cookie httpOnly não é acessível por JavaScript, o frontend descobre o usuário autenticado consultando o endpoint `GET /me` ao carregar a aplicação.

3.2. Organização do backend

O backend segue o padrão **MVC em camadas**, com responsabilidades separadas e injeção de dependência por construtor:

Camada	Diretório	Responsabilidade
Model	<code>src/models/</code>	Interfaces TypeScript que descrevem o formato das linhas das tabelas e os objetos de transferência de dados
Repository	<code>src/repositories/</code>	Único ponto de acesso ao banco. Consultas SQL parametrizadas e controle de transação
Service	<code>src/services/</code>	Regra de negócio: validações, verificação de conflito e de disponibilidade, ordenação
Controller	<code>src/controllers/</code>	Tradução entre HTTP e domínio: leitura de parâmetros, montagem da resposta
Router	<code>src/routes/</code>	Declaração dos endpoints e associação com os controllers
Middleware	<code>src/middleware/</code>	Autenticação por cookie e tratamento centralizado de erros
Utils	<code>src/utils/</code>	Funções de apoio: hash, JWT, data/hora e algoritmo de ordenação

3.3. Organização do frontend

Diretório	Conteúdo
<code>src/pages/</code>	Uma pasta por tela: Login, Home, AppointmentRegistration, AppointmentManagement e History
<code>src/components/</code>	Componentes reutilizáveis: Modal, Alerts, AppointmentForm, AppointmentTable, UpcomingAppointments e PrivateRoute

Diretório	Conteúdo
<code>src/contexts/</code>	Contexto de autenticação, que mantém o usuário logado e os estados de carregamento
<code>src/layouts/</code>	Layout do painel, com cabeçalho de navegação e botão de saída
<code>src/api/</code>	Instância do axios configurada com <code>withCredentials</code>
<code>src/types/</code>	Tipos que espelham o contrato da API
<code>src/utils/</code>	Formatação de data e hora para exibição e para os campos de formulário

3.4. Decisões técnicas relevantes

Algoritmo de ordenação (RF14). A ordenação da interface de gestão é feita por um *merge sort* implementado manualmente em `backend/src/utils/sorting.ts`. A consulta SQL que alimenta esse endpoint (`listUnsorted`) foi escrita **propositalmente sem cláusula `ORDER BY`**, de modo que a ordem apresentada ao usuário venha inteiramente do algoritmo implementado, e não do banco de dados. O frontend também não reordena a lista recebida.

Alertas não bloqueiam a gravação (RF17 e RF18). O enunciado determina que o sistema deve *gerar um alerta* em caso de conflito ou indisponibilidade. Optou-se, portanto, por registrar o agendamento e devolver os alertas junto da resposta, deixando a decisão final para o operador. Por consequência, não foi criada nenhuma restrição de exclusão mútua no banco de dados (como `EXCLUDE USING gist`), o que também permite que a massa de dados de demonstração contenha conflitos propositais.

Tratamento de data e hora. As colunas de data e hora são do tipo `TIMESTAMP` (sem fuso horário) e representam a *hora local do serviço*: uma faxina às 09:00 é sempre 09:00. Para garantir esse comportamento, o driver `pg` foi configurado para devolver o valor bruto no formato `AAAA-MM-DDTHH:mm:ss`, e a API rejeita valores que contenham indicação de fuso horário. Sem esse cuidado, um agendamento editado sem alteração de horário sofreria deslocamento de três horas a cada gravação.

Registro do histórico em transação. A gravação do agendamento e o registro correspondente na tabela de histórico ocorrem dentro da mesma transação de banco. Ou as duas operações são confirmadas, ou nenhuma é — o que impede que exista agendamento sem rastro no histórico.

4. Modelo de Dados

O banco de dados `faxina_db` é composto por sete tabelas. O Diagrama Entidade-Relacionamento é entregue separadamente, em arquivo de imagem, conforme o item 2 do enunciado.

Tabela	Finalidade	Registros
<code>usuario</code>	Operadores da empresa que acessam o sistema. Armazena o hash bcrypt da senha e o perfil (ADMIN ou OPERADOR)	3
<code>cliente</code>	Pessoas físicas e jurídicas que solicitam o serviço de faxina	3
<code>endereco</code>	Endereços vinculados a um cliente, onde o serviço será executado	3
<code>profissional</code>	Profissionais de limpeza que executam os serviços	3
<code>disponibilidade</code>	Janelas semanais de atendimento de cada profissional (dia da semana e faixa de horário)	3
<code>agendamento</code>	Agendamentos de faxina, com cliente, profissional, endereço, tipo e intervalo de data e hora	4
<code>historico_agendamento</code>	Registro de cada criação, edição e exclusão, com o operador responsável e a data da operação	4

Convenções e restrições

- A coluna `disponibilidade.dia_semana` segue a convenção do `EXTRACT(DOW)` do PostgreSQL: **0 = domingo** até **6 = sábado**. A convenção está documentada no próprio schema por meio de `COMMENT ON COLUMN`.
- Os tipos enumerados `perfil_usuario`, `tipo_faxina` e `tipo_acao` restringem os valores aceitos nas respectivas colunas.
- `historico_agendamento.agendamento_id` usa `ON DELETE SET NULL`: quando um agendamento é excluído, o registro histórico permanece, apenas perdendo a referência. Isso preserva a rastreabilidade exigida pelo enunciado.
- Restrições `CHECK` garantem que a hora de término seja posterior à de início, tanto em `agendamento` quanto em `disponibilidade`.
- Índice `idx_agendamento_prof_inicio` sobre `(profissional_id, data_hora_inicio)` apoia a verificação de conflito, executada a cada gravação.

Massa de dados de demonstração. Os agendamentos do script de população contêm conflitos propositalmente, para que os alertas dos requisitos RF17 e RF18 possam ser demonstrados sem necessidade de cadastro prévio. Os detalhes estão descritos na seção 6.3.

5. Lista de Requisitos Funcionais

Foram identificados 20 requisitos funcionais. A coluna *Origem* indica o item do enunciado que originou cada requisito; os requisitos RF19 e RF20 derivam do texto da seção DESAFIO.

Código	Nome	Descrição	Origem	Prioridade
RF01	Autenticar usuário	O sistema deve permitir que o operador acesse o sistema informando e-mail e senha. As credenciais são validadas contra a tabela <code>usuario</code> , comparando a senha informada com o hash bcrypt armazenado.	Item 4	Alta
RF02	Informar motivo da falha de autenticação	Em caso de falha no acesso, o sistema deve exibir o motivo da falha e manter o usuário na tela de autenticação, permitindo nova tentativa.	Item 4.1	Alta
RF03	Exibir nome do usuário logado	A interface principal deve exibir o nome do operador autenticado.	Item 5.1.1	Alta
RF04	Efetuar logout	O sistema deve oferecer um meio de encerrar a sessão, redirecionando o usuário à tela de autenticação.	Item 5.1.2	Alta
RF05	Acessar cadastro de agendamento	A interface principal deve oferecer um meio de acessar a interface de cadastro de agendamento.	Item 5.1.3	Alta
RF06	Acessar gestão de agendamentos	A interface principal deve oferecer um meio de acessar a interface de gestão de agendamentos.	Item 5.1.4	Alta
RF07	Listar agendamentos automaticamente	Ao acessar a interface de cadastro, o sistema deve carregar automaticamente, em formato de tabela, os agendamentos existentes no banco de dados.	Item 6.1.1	Alta
RF08	Buscar agendamentos por termo	O sistema deve oferecer um campo de busca que, após inserção e confirmação do termo, atualize a listagem exibindo apenas os registros correspondentes. A busca abrange nome do cliente, nome do profissional, tipo de faxina, observações, bairro e cidade.	Item 6.1.2	Alta
RF09	Cadastrar agendamento	O sistema deve permitir a inserção de um novo agendamento no banco de dados, informando cliente, endereço, profissional, tipo de faxina, data e hora de início e término, e observações.	Item 6.1.3	Alta
RF10	Editar agendamento	O sistema deve permitir a alteração dos dados de um agendamento existente.	Item 6.1.4	Alta

Código	Nome	Descrição	Origem	Prioridade
RF11	Excluir agendamento	O sistema deve permitir a exclusão de um agendamento existente, solicitando confirmação prévia do operador.	Item 6.1.5	Alta
RF12	Validar dados informados	O sistema deve validar os dados preenchidos na criação e na alteração de agendamentos, exibindo alertas em caso de ausência de preenchimento ou de dado inválido. As validações incluem campos obrigatórios, formato de data e hora, término posterior ao início, data não retroativa e pertencimento do endereço ao cliente selecionado.	Item 6.1.6	Alta
RF13	Retornar à interface principal	A interface de cadastro deve oferecer um meio de retornar à interface principal do sistema.	Item 6.1.7	Média
RF14	Listar agendamentos ordenados por algoritmo	A interface de gestão deve apresentar os agendamentos em ordem cronológica ou alfabética, utilizando um algoritmo de ordenação implementado no sistema. Foi adotado o merge sort, com três critérios selecionáveis: data, nome do cliente e nome do profissional.	Item 7.1.1	Alta
RF15	Selecionar agendamento e definir movimentação	O sistema deve permitir selecionar o agendamento que sofrerá movimentação e escolher entre marcação de faxina residencial ou comercial, ou ainda a alocação de outro profissional.	Item 7.1.2	Alta
RF16	Informar data e horário da movimentação	O sistema deve permitir que o operador informe a data e o horário do agendamento que está sendo realizado.	Item 7.1.3	Alta
RF17	Alertar conflito de horário	A cada gravação de agendamento, o sistema deve verificar automaticamente se o profissional já possui outro serviço no mesmo intervalo e, em caso positivo, exibir alerta ao operador.	Item 7.1.4	Alta
RF18	Alertar indisponibilidade do profissional	A cada gravação de agendamento, o sistema deve verificar automaticamente se o horário solicitado está dentro da janela de disponibilidade cadastrada para o profissional no dia da semana correspondente e, caso não esteja, exibir alerta ao operador.	Item 7.1.4	Alta
RF19	Alertar agendamentos próximos	O sistema deve destacar na interface principal os agendamentos prestes a serem realizados, considerando uma janela de 72 horas a partir do momento do acesso.	Desafio	Média

Código	Nome	Descrição	Origem	Prioridade
RF20	Registrar histórico de operações	O sistema deve registrar um histórico completo de cada agendamento, identificando o cliente, o profissional responsável, o operador que realizou a ação, o tipo de ação (criação, edição ou exclusão) e a data da operação.	Desafio	Alta

6. Descritivo de Casos de Teste de Software

6.1. Ferramentas utilizadas

Ferramenta	Aplicação nos testes
Navegador Google Chrome	Execução dos testes manuais de interface, inspeção do DOM e verificação de estilos aplicados
Ferramentas de Desenvolvedor do navegador	Acompanhamento das requisições à API, dos códigos de status HTTP e do console de erros
cURL	Testes diretos aos endpoints da API, incluindo verificação do cabeçalho <code>Set-Cookie</code> e dos códigos de erro retornados
psql (cliente PostgreSQL)	Conferência da massa de dados, verificação da persistência após cada operação e restauração do estado inicial entre baterias de teste
Compilador TypeScript (<code>tsc</code>)	Verificação estática de tipos em backend e frontend antes de cada rodada de testes

6.2. Ambiente de teste

Componente	Configuração
Sistema operacional	Microsoft Windows 11 Pro, versão 10.0.26200.8973
Banco de dados	PostgreSQL 18.3, instância local em <code>localhost:5432</code> , base <code>faxina_db</code>
Servidor de aplicação (API)	Node.js v24.11.1 executando em <code>http://localhost:5000</code>
Servidor de interface	Vite 8.2.1 executando em <code>http://localhost:3000</code>
Navegador	Google Chrome, resolução de viewport 1280 × 720
Tipo de teste	Teste manual funcional de caixa-preta, executado pela interface do sistema
Data de execução	09 de agosto de 2026

6.3. Massa de dados

Todos os casos de teste partem da base populada pelo script `backend/db/faxina_db.sql`. A massa contém três clientes, três profissionais, três endereços, três janelas de disponibilidade e quatro agendamentos. Os quatro agendamentos foram construídos para cobrir os cenários de alerta:

Id	Cliente / Profissional	Data e hora	Situação esperada
1	Helena Costa / Sonia Ferreira	10/08/2026, 09:00 às 11:00	Conflita com o agendamento 2 (mesma profissional)
2	Condomínio Vista Verde / Sonia Ferreira	10/08/2026, 10:00 às 12:00	Conflita com o agendamento 1 (mesma profissional)

Id	Cliente / Profissional	Data e hora	Situação esperada
3	Escritório Alfa Contábil / Carlos Mendes	10/08/2026, 09:00 às 12:00	Segunda-feira; o profissional só atende às terças — indisponibilidade
4	Helena Costa / Patrícia Lima	14/08/2026, 08:00 às 12:00	Sexta-feira, dentro da janela de atendimento — sem alertas

Janelas de disponibilidade cadastradas: Sonia Ferreira às segundas, das 08:00 às 18:00; Carlos Mendes às terças, das 08:00 às 17:00; Patrícia Lima às sextas, das 07:00 às 15:00.

6.4. Casos de teste

Identificação	Procedimento e resultado
CT01 RF01 Autenticar usuário	Pré-condição: base populada; usuário não autenticado. Procedimento: acessar a tela de login, informar e-mail e senha válidos e acionar "Entrar". Dados de entrada: <code>gustavo@faxinaja.com.br</code> / <code>admin123</code> Resultado esperado: autenticação concluída, cookie de sessão criado e redirecionamento à interface principal. Resultado obtido: o sistema autenticou o operador e exibiu a interface principal com a mensagem "Bem-vindo, Gustavo". Status: Aprovado
CT02 RF02 Motivo da falha de autenticação	Pré-condição: usuário não autenticado. Procedimento: executar três tentativas de acesso — (a) com os campos em branco; (b) com e-mail válido e senha incorreta; (c) com e-mail não cadastrado. Dados de entrada: (a) campos vazios; (b) <code>gustavo@faxinaja.com.br</code> / <code>senhaerrada</code>; (c) <code>naoexiste@faxinaja.com.br</code> / qualquer senha. Resultado esperado: (a) mensagem "Informe o e-mail"; (b) e (c) mensagem "E-mail ou senha incorretos", idêntica nos dois casos; em todos, permanência na tela de login. Resultado obtido: as três mensagens foram exibidas conforme o esperado e a tela de login foi mantida em todas as tentativas. Status: Aprovado
CT03 RF03 Nome do usuário logado	Pré-condição: operador autenticado (CT01). Procedimento: observar a interface principal e o cabeçalho de navegação. Resultado esperado: exibição do nome do operador autenticado. Resultado obtido: a interface exibiu "Bem-vindo, Gustavo" no corpo e "Olá, Gustavo" no cabeçalho. Após atualização da página (F5), o nome permaneceu, confirmando a recuperação da sessão pelo cookie. Status: Aprovado
CT04 RF04 Efetuar logout	Pré-condição: operador autenticado. Procedimento: acionar o botão "Sair" no cabeçalho. Resultado esperado: encerramento da sessão e redirecionamento à tela de login. Resultado obtido: a sessão foi encerrada e o sistema apresentou a tela de login. Status: Aprovado

Identificação	Procedimento e resultado
CT05 RF05 Acessar cadastro de agendamento	Pré-condição: operador autenticado, na interface principal. Procedimento: acionar o cartão "Cadastro de Agendamento". Resultado esperado: abertura da interface de cadastro de agendamento. Resultado obtido: o sistema apresentou a tela "Cadastro de Agendamento". Status: Aprovado
CT06 RF06 Acessar gestão de agendamentos	Pré-condição: operador autenticado, na interface principal. Procedimento: acionar o cartão "Gestão de Agendamentos". Resultado esperado: abertura da interface de gestão de agendamentos. Resultado obtido: o sistema apresentou a tela "Gestão de Agendamentos". Status: Aprovado
CT07 RF07 Listagem automática	Pré-condição: base com os quatro agendamentos do script de população. Procedimento: acessar a interface de cadastro de agendamento e observar a tabela, sem acionar nenhum comando. Resultado esperado: exibição automática dos quatro agendamentos, com cliente, profissional, tipo, início e término. Resultado obtido: a tabela foi carregada automaticamente com os quatro registros e todas as colunas preenchidas. Status: Aprovado
CT08 RF08 Buscar por termo	Pré-condição: interface de cadastro aberta, com quatro registros listados. Procedimento: digitar um termo no campo de busca e acionar "Buscar"; em seguida, limpar o campo e buscar novamente. Dados de entrada: termo Carlos ; depois campo vazio. Resultado esperado: com o termo, exibição apenas dos agendamentos do profissional Carlos Mendes; com o campo vazio, retorno da listagem completa. Resultado obtido: a busca por "Carlos" retornou apenas um registro (Escritório Alfa Contábil / Carlos Mendes) e a limpeza do campo restaurou os quatro registros. Status: Aprovado
CT09 RF09 Cadastrar agendamento	Pré-condição: interface de cadastro aberta, com quatro registros listados. Procedimento: acionar "Novo agendamento", preencher o formulário e acionar "Salvar". Dados de entrada: cliente Helena Costa; endereço Rua das Palmeiras, 120; profissional Patrícia Lima; tipo Residencial; início 21/08/2026 09:00; término 21/08/2026 10:00. Resultado esperado: agendamento gravado, janela fechada e tabela atualizada com cinco registros. Resultado obtido: o registro foi criado, a janela foi encerrada e a listagem passou a exibir cinco agendamentos, incluindo o novo em 21/08/2026. Observou-se também que o campo de endereço só é habilitado após a escolha do cliente, sendo carregado conforme o cliente selecionado. Status: Aprovado

Identificação	Procedimento e resultado
CT10 RF10 Editar agendamento	Pré-condição: existência do agendamento criado em CT09. Procedimento: acionar "Editar" na linha correspondente, conferir o preenchimento do formulário e acionar "Salvar" sem alterar nenhum campo. Resultado esperado: formulário pré-preenchido com os dados atuais e, após a gravação, manutenção integral dos valores — em especial o horário, que não deve sofrer deslocamento. Resultado obtido: o formulário abriu preenchido com 21/08/2026 09:00 e, após a gravação, a listagem manteve exatamente 21/08/2026 09:00. Status: Aprovado
CT11 RF11 Excluir agendamento	Pré-condição: existência do agendamento criado em CT09. Procedimento: acionar "Excluir" na linha correspondente e confirmar a operação na caixa de diálogo. Resultado esperado: solicitação de confirmação antes da exclusão; após confirmar, remoção do registro e atualização da tabela. Resultado obtido: o sistema solicitou confirmação; ao cancelar, o registro foi preservado; ao confirmar, o registro foi removido e a listagem retornou a quatro agendamentos. Status: Aprovado
CT12 RF12 Validar dados informados	Pré-condição: interface de cadastro aberta. Procedimento: acionar "Novo agendamento" e tentar salvar o formulário sem preencher nenhum campo. Resultado esperado: bloqueio da gravação, permanência da janela aberta e sinalização dos campos obrigatórios; nenhum registro criado. Resultado obtido: a gravação foi bloqueada, a janela permaneceu aberta e a listagem continuou com quatro registros. Em teste complementar pela API, o envio de data em formato inválido retornou a mensagem "O campo data_hora_inicio deve estar no formato AAAA-MM-DDTHH:mm" com código HTTP 400. Status: Aprovado
CT13 RF13 Retornar à interface principal	Pré-condição: interface de cadastro aberta, imediatamente após a execução de uma operação de gravação ou exclusão. Procedimento: acionar o comando "Voltar". Resultado esperado: retorno imediato à interface principal. Resultado obtido: na primeira execução o comando não respondeu (defeito D-01, descrito na seção 6.6). Após a correção, o retorno à interface principal ocorreu imediatamente, mesmo com notificação de sucesso ainda visível na tela. Status: Aprovado após correção
CT14 RF14 Ordenação por algoritmo	Pré-condição: interface de gestão aberta. Procedimento: alternar o seletor de critério entre "data", "cliente" e "profissional", observando a ordem da lista a cada troca. Resultado esperado: reordenação da lista conforme o critério escolhido, em ordem cronológica ou alfabética crescente. Resultado obtido: por data, a lista foi ordenada de 10/08 a 14/08; por cliente, na ordem Condomínio Vista Verde, Escritório Alfa Contábil e Helena Costa; por profissional, na ordem Carlos Mendes, Patrícia Lima e Sonia Ferreira. A ordenação é produzida pelo merge sort da aplicação, sendo a consulta SQL correspondente desprovida de <code>ORDER BY</code>. Status: Aprovado

Identificação	Procedimento e resultado
CT15 RF15 Selecionar agendamento e definir movimentação	Pré-condição: interface de gestão aberta. Procedimento: selecionar um agendamento na lista e observar o painel de movimentação; alterar o tipo de faxina e o profissional alocado. Resultado esperado: painel habilitado após a seleção, com os dados atuais do agendamento e possibilidade de escolher entre residencial e comercial e de alocar outro profissional. Resultado obtido: antes da seleção o painel exibia a orientação "Selecione um agendamento na lista ao lado"; após a seleção, apresentou o tipo e o profissional atuais, permitindo a alteração de ambos. Ao trocar o profissional, o sistema passou a exibir a janela de disponibilidade correspondente. Status: Aprovado
CT16 RF16 Informar data e horário	Pré-condição: agendamento selecionado na interface de gestão. Procedimento: observar os campos de data e hora de início e término e alterar seus valores. Resultado esperado: campos preenchidos com o horário atual do agendamento e editáveis pelo operador. Resultado obtido: os campos apresentaram os valores corretos do agendamento selecionado e aceitaram alteração, sendo os novos valores persistidos após a gravação. Status: Aprovado
CT17 RF17 Alerta de conflito de horário	Pré-condição: interface de gestão aberta; massa de dados original. Procedimento: selecionar um agendamento e alocar um profissional que já possua outro serviço no mesmo intervalo; acionar "Salvar movimentação". Dados de entrada: agendamento do Condomínio Vista Verde em 10/08/2026, das 10:00 às 12:00, realocado para Carlos Mendes, que já atende no mesmo intervalo. Resultado esperado: gravação efetuada e exibição do alerta "Conflito de horário". Resultado obtido: o sistema gravou a movimentação e exibiu o alerta "Conflito de horário — Este profissional já possui outro serviço agendado neste intervalo de horário." Status: Aprovado
CT18 RF18 Alerta de indisponibilidade	Pré-condição: interface de gestão aberta; massa de dados original. Procedimento: selecionar um agendamento cujo dia da semana esteja fora da janela de atendimento do profissional alocado e acionar "Salvar movimentação". Dados de entrada: agendamento de Carlos Mendes em 10/08/2026 (segunda-feira), sendo sua disponibilidade cadastrada apenas às terças, das 08:00 às 17:00. Resultado esperado: gravação efetuada e exibição do alerta "Profissional indisponível". Resultado obtido: o sistema gravou a movimentação e exibiu o alerta "Profissional indisponível — O profissional não possui disponibilidade cadastrada para este dia e horário.", acompanhado da informação "Disponível: Terça 08:00–17:00". Em cenário que reunia as duas condições, os dois alertas foram exibidos simultaneamente. Status: Aprovado
CT19 RF19 Agendamentos próximos	Pré-condição: operador autenticado; base com agendamentos dentro da janela de 72 horas. Procedimento: acessar a interface principal e observar o painel de destaque. Resultado esperado: exibição dos agendamentos cuja data de início esteja nas próximas 72 horas, com data, hora, cliente e profissional. Resultado obtido: o painel "Agendamentos nas próximas 72 horas" listou os três agendamentos de 10/08/2026, omitindo corretamente o de 14/08/2026, fora da janela. Status: Aprovado

Identificação	Procedimento e resultado
CT20 RF20 Histórico de operações	Pré-condição: operações de criação, edição e exclusão executadas nos casos CT09, CT10 e CT11. Procedimento: acessar a interface de histórico e conferir os registros. Resultado esperado: um registro para cada operação realizada, contendo o tipo de ação, o cliente, o profissional, o operador responsável e a data e hora. Resultado obtido: o histórico apresentou os registros de criação, edição e exclusão referentes aos casos anteriores, todos atribuídos ao operador Gustavo, além dos quatro registros de criação da massa inicial. A linha correspondente à exclusão foi preservada mesmo após a remoção do agendamento, confirmando o comportamento de <code>ON DELETE SET NULL</code>. Status: Aprovado

6.5. Resumo dos resultados

Indicador	Resultado
Casos de teste planejados	20
Casos de teste executados	20 (100%)
Casos aprovados na primeira execução	19 (95%)
Casos aprovados após correção	1 (CT13)
Casos reprovados ao final	0
Defeitos registrados	2 (ambos corrigidos e reavaliados)

6.6. Defeitos encontrados e corrigidos

Código	Título	Descrição, causa e correção
D-01	Comando "Voltar" não responde após uma operação (CT13 / RF13)	Sintoma: ao excluir ou gravar um agendamento e acionar "Voltar" em seguida, o comando não produzia efeito. Causa: o contêiner de notificações estava posicionado no canto superior direito, sobrepondo a área do comando. O elemento da notificação permanece na árvore do documento durante a animação de saída, continuando a interceptar os cliques mesmo após deixar de ser visível. Correção: reposicionamento do contêiner de notificações para o canto inferior direito. Reteste: o cenário original foi reexecutado — exclusão seguida de acionamento imediato de "Voltar" — e o retorno ocorreu de imediato.

Código	Título	Descrição, causa e correção
D-02	Deslocamento de três horas ao editar agendamento (CT10 / RF10)	Sintoma: ao abrir um agendamento para edição e gravá-lo sem alterar o horário, o horário avançava três horas a cada gravação. Causa: o driver de banco convertia as colunas <code>TIMESTAMP</code> em objetos de data do JavaScript, que eram serializados em UTC na resposta da API. O valor devolvido ao cliente, portanto, diferia do valor armazenado, e ao ser reenviado provocava nova conversão. Correção: configuração do driver para devolver o valor bruto da coluna, no formato de hora local <code>AAAA-MM-DDTHH:mm:ss</code>, e rejeição, pela API, de valores que contenham indicação de fuso horário. Reteste: o ciclo de leitura e gravação foi repetido pela interface e pela API, mantendo o horário inalterado em ambos.

7. Instruções de Execução

Passo 1 — Criar e popular o banco de dados

Com o PostgreSQL 18 em execução, executar o script de criação a partir da raiz do projeto:

```
psql -U postgres -f backend/db/faxina_db.sql
```

O script remove a base existente, recria a base `faxina_db`, cria os tipos, as tabelas e os índices, e insere a massa de dados. Caso exista alguma sessão conectada à base (por exemplo, uma janela do pgAdmin), o comando de remoção falhará; basta encerrar a conexão e executar novamente.

Passo 2 — Configurar as variáveis de ambiente

Na pasta `backend`, copiar o arquivo `.env.example` para `.env` e preencher os valores conforme o ambiente local, em especial a senha do PostgreSQL:

Variável	Finalidade
<code>PORT</code>	Porta da API (padrão 5000)
<code>CORS_ORIGIN</code>	Endereço da interface autorizado a consumir a API
<code>CHAVE_ACESSO</code>	Chave secreta de assinatura do token de acesso
<code>COOKIE_NOME</code>	Nome do cookie que transporta o token
<code>DB_HOST</code> , <code>DB_PORT</code>	Endereço e porta do servidor PostgreSQL
<code>DB_USUARIO</code> , <code>DB_SENHA</code>	Credenciais de acesso ao banco
<code>DB_NOME</code>	Nome da base de dados (<code>faxina_db</code>)

Passo 3 — Iniciar a API

Na pasta `backend`: `npm install` e, em seguida, `npm run dev`. A API passa a responder em `http://localhost:5000`.

Passo 4 — Iniciar a interface

Na pasta `frontend`: `npm install` e, em seguida, `npm run dev`. A interface fica disponível em `http://localhost:3000`.

A interface deve necessariamente ser servida na porta **3000**, pois é o endereço autorizado na configuração de CORS da API. Executá-la em outra porta faz com que todas as requisições sejam bloqueadas pelo navegador.

Passo 5 — Acessar o sistema

Abrir `http://localhost:3000` no navegador e autenticar com uma das credenciais relacionadas na seção 8.

8. Credenciais de Acesso

O script de população cria três operadores. As senhas são armazenadas exclusivamente como hash bcrypt com fator de custo 12; os valores em texto abaixo são fornecidos apenas para avaliação do sistema.

E-mail	Senha	Perfil	Nome
<code>gustavo@faxinaja.com.br</code>	<code>admin123</code>	ADMIN	Gustavo
<code>mariana.duarte@faxinaja.com.br</code>	<code>operador123</code>	OPERADOR	Mariana Duarte
<code>rafael.nunes@faxinaja.com.br</code>	<code>operador123</code>	OPERADOR	Rafael Nunes

Os hashes podem ser regerados, caso as senhas sejam alteradas, por meio do comando `npm run hash` executado na pasta `backend`.

Para demonstrar os alertas dos requisitos RF17 e RF18 imediatamente após a instalação, basta acessar a interface de gestão, selecionar o agendamento do Escritório Alfa Contábil (10/08/2026) e acionar "Salvar movimentação": o alerta de indisponibilidade será exibido, pois o profissional Carlos Mendes atende somente às terças-feiras. Para o alerta de conflito, selecionar o agendamento de Helena Costa com Sonia Ferreira, no mesmo dia, cujo intervalo se sobrepõe ao do Condomínio Vista Verde.